B03 | 怎样只把可用回应交给后续代码

本章换到调用方：**Python 值 → HTTPX 请求 → 检查回应 → 只交回可用摘要。** FastAPI 接收处理，HTTPX 主动发送；不是让两个程序直接调用彼此的 Python 函数名。

请求发给谁，值放在哪里？

当前问题	关系或短用法	记到什么程度
怎样把一次调用变成请求？	`client.post("/summaries", ...)` 选 POST 和 path，`base_url` 提供服务地址；`json={"text": text}` 编成 JSON body，并设置相应内容类型。	逐渐用熟“地址、方法、内容”的组合，端口和样例文字不背。
`params=`、`json=`、`headers=` **能互换吗？**	分别放 URL query、JSON body、消息头；放哪里由当前 API 约定决定。	**直接分清三个位置。** 具体字段查接口，不能只因都在传值就互换。
收到的 `.text` 和 `.json()` 差在哪？	`response.text` 是解码后的正文文字；`response.json()` 尝试解析成 Python 值。	认熟“文字 → 值”的方向；解析不会顺带证明成功或字段正确。

`Accept` 表达希望收到的类型，仍要核对实际回应。本章 `X-Request-ID` 在 header 中关联记录，不是正文，也不是工单编号；B04 再把它串到两端。

回看：「先把一次普通调用读完整」「`Response` 到手，还没有到可以使用摘要的时候」。

回应到手后，怎样变成可以使用的结果？

本章的先后关系是：**成功状态 → 约定的内容类型 → JSON 能解析 → 类型和字段可用 → 返回摘要。** 过了前一关，只允许走到下一关。

当前问题	关系或短用法	记到什么程度
有 `Response`，是否已经成功？	失败也会有回应。`response.raise_for_status()` 对非 `2xx` 抛出 `HTTPStatusError`；失败正文也可能是合法 JSON。	**拿到回应不等于拿到可用结果。**若合同要求精确 200 或 201，还需单独比较。
内容类型怎样判定？	本例要求 `application/json`，先处理大小写、空白和分号后的参数，再精确比较；缺失或其他类型都拒绝。	理解“规范化后精确比较”，整行字符串配方查用，不记成“包含 json 就行”。
声明 JSON 就能解析吗？	声明正确，body 仍可能损坏；`response.json()` 只处理解析。反过来，`text/plain` 的正文恰好能解析，也未满足本例合同。	**声明、JSON 语法、接口合同分开检查。**
能解析，为什么还不能直接 `.strip()`？	先确认顶层是 `dict`，再取 `summary`；确认摘要是 `str`，才能去两端空白，再拒绝空结果。	顺序取决于下一步操作的前提；列表、缺键、数字、只有空格都不满足本例的摘要约定。
哪一步才交给后面？	所有检查通过才 `return summary`；在 `summary = request_summary(...)` 中，右侧成功返回，左侧本次赋值才完成。	能沿返回追踪；失败时残留的旧摘要不能冒充本次结果。

查用时，规范化就是取第一个分号前的部分，去两端空白、转小写。例如 `Application/JSON; Charset=UTF-8` 得到 `application/json`。这是本接口的合同，其他接口可约定不同类型。HTTPX 默认不跟随重定向，本章 302 也在状态检查处停止；`raise_for_status()` 本身不负责跳转。

回看：「先检查 status，因为失败回应也可能带着 JSON」「内容类型、JSON 语法和字段，是三件不同的事」「把这些检查接成一个有明确出口的函数」。

同样叫失败，最早停在哪里？

当前问题	关系或短用法	记到什么程度
没取得完整回应，就先去读 status 吗？	transport 路线在 `client.post(...)` 抛出 `RequestError` 或子类，本次 `response = ...` 没完成，状态检查尚未开始。	先认失败层，不读尚未取得的回应；**客户端没拿到，不证明对方没执行。**
收到 503 后停止，与连接失败有什么不同？	status 路线已有回应，`raise_for_status()` 抛 `HTTPStatusError`，可从 `exc.response` 查看那份回应。	分清“没取得完整回应”和“回应报告非成功”，不全叫网络断了。
200 后仍失败，查哪里？	content 路线可能停在媒体类型、解析、顶层类型、字段或非空检查；小例用 `ValueError` 表达坏内容。	找最早未通过的一关，不把所有问题归给 `.json()`。
单独处理超时，`except` 顺序为什么重要？	`TimeoutException` 属于 `RequestError`，先接具体超时，再接宽泛请求错误，否则超时会被先接走。	能解释分类为何丢失；完整异常名和分支查正文，不成套背诵。

本章用 `try/except/else` 分开失败处理和成功显示。接住异常不等于补出了成功值；无关计算不要混入同一个 `try`，免得其 `ValueError` 也被误报为供应商坏内容。

回看：「同样一句“调用失败”，最早停止的位置可能不同」。

等待、复用与受控测试，各自改变哪一段？

当前问题	关系或短用法	记到什么程度
`timeout=1.0` 是整次调用的一秒总限时吗？	connect 等建连接；read 等收到下一块响应数据；write 等写出下一块请求数据；pool 等可用连接。它们不是同一只总秒表。	**认清等待位置，不能承诺整次一秒内结束。** 数值和异常子类全名可查。
反复调用，为什么保留一个 Client？	`httpx.post(...)` 适合一次临时调用；保留 `Client` 可复用连接、集中配置。每次 request/response 仍是独立消息。	按使用方式选入口；每轮重建 Client 不等于跨轮复用。
Client 用完，谁收尾？	`with httpx.Client(...) as client:` 离开区间时关闭资源，包括异常退出；长运行应用也可持有它并安排停止时关闭。	认熟“建立者安排收尾”，关闭客户端不等于停止远端服务。
不启动服务，`MockTransport` 测到什么？	它接到 HTTPX 构造好的 request，返回受控 Response 或抛异常；可核对请求内容和客户端处理分支。	理解替换点；主动抛 `ReadTimeout` 不证明真实等过一秒，也不证明网络接通。
`SupplierClient` 为什么让调用变短？	它集中发送和回应检查，成功交回摘要，失败翻译为本应用异常；检查没有消失，只移到适配层。	多处需要统一合同才更有价值；一次临时调用可直接用 HTTPX，不为形式加包装。

查配置时，`httpx.Timeout(1.0, connect=2.0)` 表示连接等待两秒，其余等待一秒。超时结束等待，不取消对方已做的工作，也不自动重试。`trust_env=False` 是本地练习隔离代理环境影响的选择，不是所有项目必选项；真实凭据来自配置，不放 URL 或日志。

适配层自定义的失败名称在 B04 与外层回应一起查，不另背一套词表。`raise ... from exc` 保留底层原因，不抵消失败。应用创建的客户端由应用收尾；测试传入的替身由提供方管理。

回看：「多次调用同一个服务，为什么保留一个 Client」「一秒 timeout 到底限制哪段等待」「不启动网络服务，也能检查客户端如何面对回应」「已有的 SupplierClient 包装，究竟增加了什么」。

B04 把调用方放进另一个服务内部：**本地 API 接收用户请求，又去请求供应商。** 两次交换各有回应，不能混成一次。